



C# - Introduction to .net

- Introduction to .Net Framework
- Versions of the Framework
- CLR
- Intermediate Language (IL)
- JIT Compilation
- Garbage Collection
- AppDomain Management
- CLS
- CTS
- FCL
- Using ILDASM
- Assemblies and their structure
- shared assembly
- Managed and Unmanaged Code
- Introduction to Visual Studio
- Debugging



- .NET framework is a complete development and execution environment. That allows the developer to develop, run, and deploy the following application.
 - ◆ Console application.
 - ◆ Windows application
 - ◆ Web application
 - ◆ Service-oriented application (WCF)
 - ◆ Web service
 - ◆ Windows service
 - ◆ WPF, WWF, etc.
- .NET Framework supports the object-oriented programming model for multiple languages such as C#, VB, VC++, J#, etc.
- It supports language interoperability. Interoperability implies that each language can use the code written in some other language.
- It provides a common set of tools and enables easy integration of modules developed with one another.
- It also enables the developers to create a shareable component to be used in distributed computing architecture.

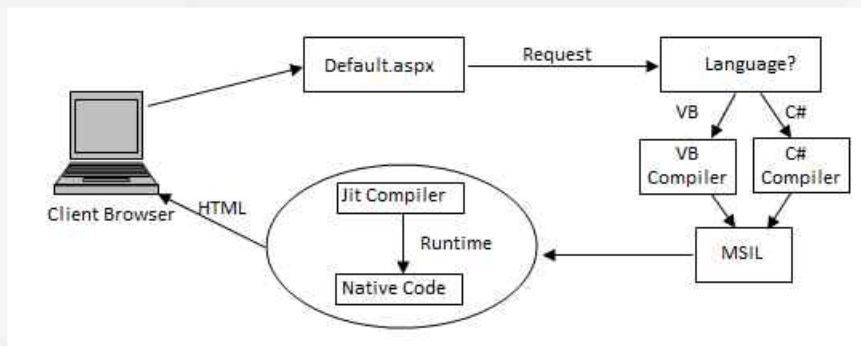
- .NET Framework was designed to fulfill the following goals.
 1. Interoperability
 2. Cross-platform Support
 3. Language Independence
 4. Base class library.
 5. Easy development
 6. Security

■ Interoperability

.NET Framework supports interoperability between new applications and existing applications. It implies that the code written in one language can be used in some other language.

■ Cross-Platform support

Each .NET compatible language, such as C#, VB, VC++ provides its own compiler to compile code to MSIL. After that, the common runtime engine compiles MSIL code to native code with the help of the Just In Time (JIT) compiler and then run the application.



■ **Language Independence**

.NET framework provides multiple language support by using a feature known as a common type system. That is built into common language runtime.

■ **Base Class Library**

The base class library provides the fundamental building block for any application you develop. These applications could be windows, web, console, or service-oriented. The BCL mostly serves as the main point of interaction with the CLR.

■ **Easy Deployment**

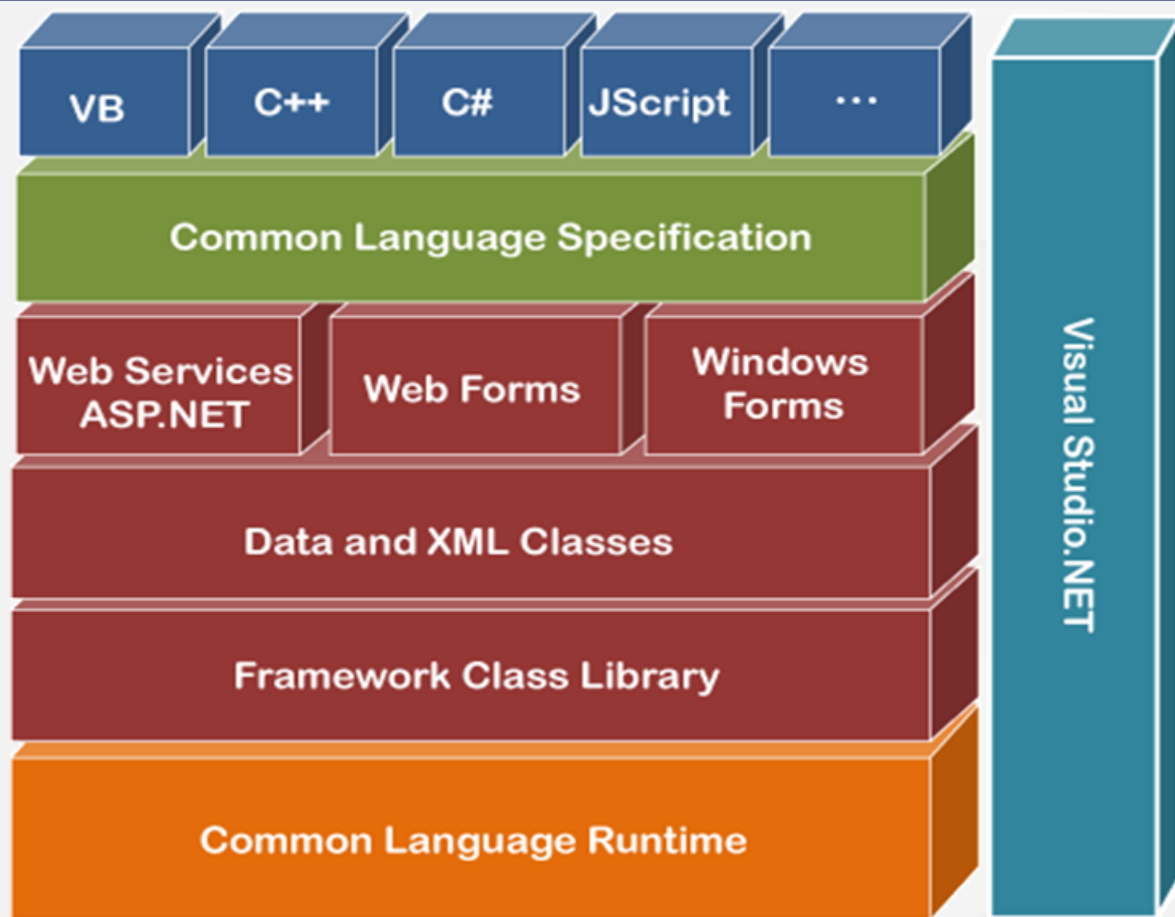
.NET Framework makes the task of employment easier. In most cases, installing an application along with a component on the target computer is easier.

■ Security

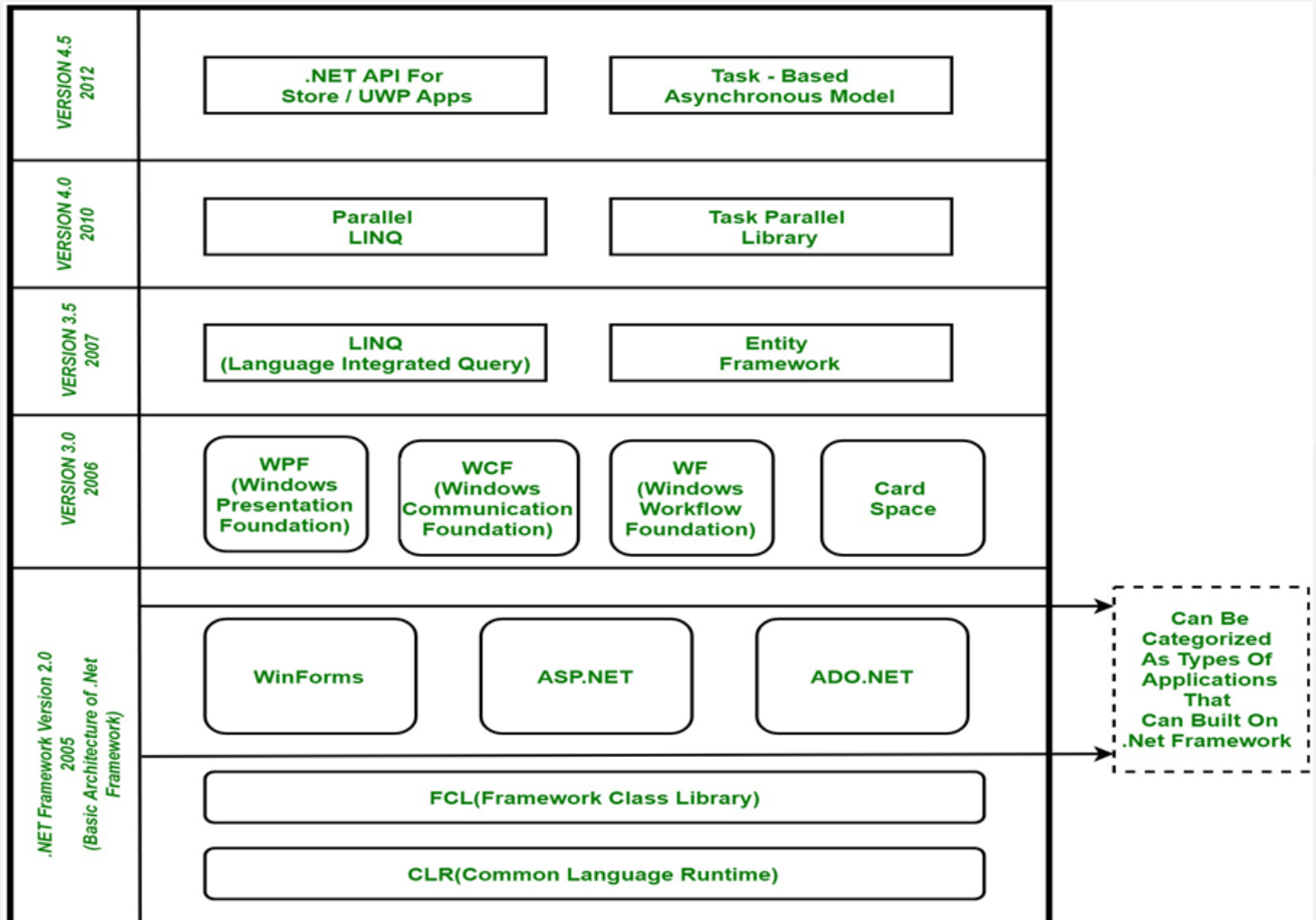
.NET Framework provides a security model that allows you to authorize and authenticate the user.

- Authentication verifies the identity of a user or service.
- Authorization determines their access rights.
- Although the two terms sound alike, they play separate but equally essential roles in securing applications and data.
- Combined, they determine the security of a system.
- Authentication means to confirm your own identity, while authorization means to grant access to the system. In simple terms, authentication verifies who you are, while authorization verifies what you have access to.

Component of .NET Framework



Versions of the Framework



- There are following components of .NET Framework:
 1. CLR (Common Language Runtime)
 2. CTS (Common Type System)
 3. BCL (Base Class Library)
 4. CLS (Common Language Specification)
 5. FCL (Framework Class Library)
 6. .NET Assemblies
 7. XML Web Services
 8. Window Services

- CLR stands for "Common Language Runtime." It is the execution engine for the .NET Framework, responsible for running programs that have been compiled for the .NET Framework.
- It is an important part of a .NET framework that works like a virtual component of the .NET Framework to executes the different languages program like c# , Visual Basic, etc.
- A CLR also helps to convert a source code into the byte code, and this byte code is known as CIL (Common Intermediate Language) or MSIL (Microsoft Intermediate Language).
- After converting into a byte code, a CLR uses a JIT compiler at run time that helps to convert a CIL or MSIL code into the machine or native code.

- **There are several components of CLR, which are as follows:**
 1. CTS
 2. MSIL
 3. Execution Support Function
 4. Security
 5. Garbage collection
 6. Classloader
 7. Memory layout
 8. Exception handling
 9. Debugging support

1. Loading and execution of the program.
2. Memory Isolation for application.
3. Verification of type safety.
4. Compilation of MSIL to native code.
5. Providing Metadata.
6. Interoperability with other systems.
7. Managing exception and error.
8. Debugging and profiling.

- **MSIL (Microsoft Intermediate Language) OR Intermediate Language(IL)**
- IL stands for Intermediate Language.
- It is also known as MSIL (Microsoft Intermediate Language) or CIL (Common Intermediate Language).
- All the .NET language such as C#, J#, VB uses their own compiler.
- The compiler compiles the code into IL and after that common runtime engine converts the IL code into native code with the help of the JIT compiler.
- The intermediate code is common for all languages and it is understandable in a .NET environment. This intermediate code is known as IL or MSIL.

- In the .NET framework, Just-In-Time (JIT) compilation is the process of compiling intermediate language (IL) code into native machine code at the time of execution. This is in contrast to ahead-of-time (AOT) compilation, where the code is compiled before it is executed.
- JIT compilation allows for efficient code execution because it only compiles the code that is actually being executed, rather than the entire program. It also allows for more flexibility because the IL code can be run on any machine that has a .NET runtime, regardless of the underlying hardware architecture.
- JIT compilation is performed by the .NET runtime's JIT compiler, which is responsible for converting IL code into native code that can be executed by the processor. The native code is then stored in memory and reused for subsequent executions of the same IL code, to avoid the overhead of recompiling the code every time it is executed.
- Overall, JIT compilation is an important part of the .NET runtime and helps to ensure efficient and flexible execution of .NET code.

- CTS (**Common Type System**) is a component of CLR (Common Language Runtime) through which the .NET Framework provides support for multiple languages.
- Because it contains a type system, that is common across all the languages.
- Two common types of system languages do not require type conversion when calling the code written in another language.
- CTS provide a base set of data type for all languages supported by .NET framework.
- This means that the size of the integer and the long variable are the same across all languages.

- It is a subset of common type system (CTS) that defines a set of rules and regulations which should be followed by every language that comes under the .net framework.
- In other words, a CLS language should be cross-language integration or interoperability. For example, in C# and VB.NET language, the C# language terminate each statement with semicolon, whereas in VB.NET it is not end with semicolon, and when these statements execute in .NET Framework, it provides a common platform to interact and share information with each other.

- FCL stands for "Framework Class Library." It is a library of classes that provides a wide range of functionality for building applications using the .NET Framework.
- The FCL is a comprehensive library that includes classes for tasks such as reading and writing files, connecting to databases, displaying graphics, and handling security. It is built on top of the Base Class Library (BCL), which provides lower-level functionality such as input/output (I/O) and data access.
- The FCL is an important part of the .NET Framework, as it provides developers with a wide range of tools and functionality that they can use to build applications. It is designed to be easy to use and is highly extensible, allowing developers to build custom functionality on top of it.
- The FCL is an integral part of the .NET Framework, and it is included with every version of the .NET Framework.

■ Key Components of FCL

1. Object type
2. Implementation of data structure
3. Base data types
4. Garbage collection
5. Security and database connectivity
6. Creating common platform for window and web-based application.

- In C#, garbage collection is the process of automatically freeing up memory that is no longer being used by the program.
- This is important because it allows a program to dynamically allocate objects without having to worry about manually deallocating them when they are no longer needed. The garbage collector runs in the background and periodically checks for objects that are no longer being used, and it reclaims the memory used by those objects.
- Garbage collection prevents memory leaks during the execution of programs.
- It is a low priority process that manages the allocation and deallocation of memory in your application.
- It checks for unreferenced variables and objects.
- If GC finds any object that is no longer used by the application, it frees up the memory of that object.

- **Generation 0** : All the short-lived objects such as temporary variables are contained in the generation 0 of the heap memory. All the newly allocated objects are also generation 0 objects implicitly unless they are large objects. In general, the frequency of garbage collection is the highest in generation 0.
- **Generation 1** : If space occupied by some generation 0 objects that are not released in a garbage collection run, then these objects get moved to generation 1. The objects in this generation are a sort of buffer between the short-lived objects in generation 0 and the long-lived objects in generation 2.
- **Generation 2** : If space occupied by some generation 1 objects that are not released in the next garbage collection run, then these objects get moved to generation 2. The objects in generation 2 are long lived such as static objects as they remain in the heap memory for the whole process duration.

- The GC.Collect() method is used to force the garbage collector to perform a collection, attempting to reclaim all unused memory. This method is part of the System namespace and is defined in the System.GC class.
- GC has changed a bit in the .NET framework 4.0. It contains the following overload method,

GC.collect ()

GC.collect (int)//Generation

- In C#, namespaces are used to logically arrange classes, structs, interfaces, enums and delegates.
- The namespaces in C# can be nested. That means one namespace can contain other namespaces also.
- The .NET framework already contains number of standard namespaces like System, System.Net, System.IO etc.
- In addition to these standard namespaces the user can define their own namespaces.
- **Declaring a Namespace**

The C# language provides a keyword namespace to create a user-defined namespace.

```
namespace <namespace_name>
{
    // Classes and/or structs and/or enums etc.
}
```

```
// defining the namespace name1
namespace name1
{
    // C1 is the class in the namespace name1
    class C1
    {
        // class code
    }
}
```

Accessing the Members of Namespace

The members of a namespace are accessed by using dot(.) operator. A class in C# is fully known by its respective namespace.

Syntax:

[namespace_name].[member_name]

- It is not possible to use any access specifiers like private, public etc with a namespace declarations.
- The namespaces in C# implicitly have public access and this is not modifiable.
- The namespace allows only public and internal elements as its members. The default is internal.
- Two classes with the same name can be created inside 2 different namespaces in a single program.
- Inside a namespace, no two classes can have the same name.
- In C#, the full name of the class starts from its namespace name followed by *dot(.)* operator and the class name, which is termed as the fully qualified name of the class.

- C# provides a keyword “using” which help the user to avoid writing fully qualified names again and again. The user just has to mention the namespace name at the starting of the program and then he can easily avoid the use of fully qualified names.

- **Syntax:**

`using [namespace_name][.][sub-namespace_name];`

Example:

```
// predefined namespace name  
using System;
```

```
// user-defined namespace name  
using name1
```

```
// namespace having subnamespace  
using System.Collections.Generic;
```

- An Assembly is a basic unit of application **deployment and versioning**.
- An Assembly is also called the building block of a .Net application.
- An Assembly is either a .exe or .dll file.
- These files are shipped along with the software.
- An assembly contains MSIL, Metadata file are required to execute a .NET program successfully.
- **Difference Between dll and exe**
- In DLL, it uses code reusability, whereas in exe it contains only output file/ or application.
- DLL file can't be open, whereas exe file can be open.
- DLL file can't be run individually, whereas in exe, it can run individually.
- In DLL file, there is no main method, whereas exe file has main method

- **There are three types of assemblies in C#:**
- **Private assemblies:** These are assemblies that are used only by a single application. They are stored in the application's local folder or subfolder, and are not intended to be shared with other applications.
- **Shared assemblies:** These are assemblies that are intended to be shared among multiple applications. They are stored in the global assembly cache (GAC), which is a special folder on the computer where shared assemblies are stored. Shared assemblies are strong-named, which means they have a unique name and version number.
- **Satellite assemblies:** These are assemblies that contain resources (such as images and localization strings) for a specific language or culture. They are used to support multiple languages in an application, and are stored in a subfolder within the application's local folder.

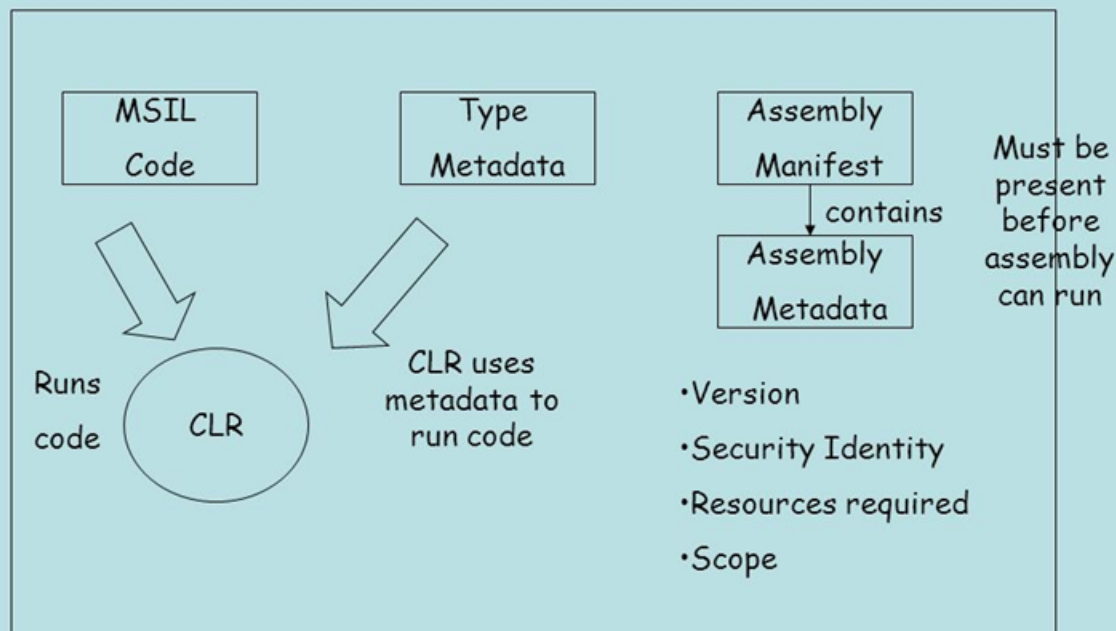
- While using a shared assembly, to avoid assembly being overwritten by the different or same assembly, Shared assembly is placed in a special directory in the file system are known as global assembly Cache.
- GAC is a central repository (cache) in a system in which assemblies are registered to share among various applications that execute on local or remote machines.
- .NET Framework provides the GAC tool (**gacutil.exe utility**), which is used to view and change the content of the GAC of a system.
- Adding new assemblies to GAC and removing assemblies from GAC are some of the tasks that can be performed using the **gacutil.exe** utility.
- GAC can contain multiple versions of the same .NET assembly.
- CLR checks GAC for a requested assembly before using the information of configuration files.

- A strong-named assembly consists of a unique version number that can only exist more than once with the same name.
- The GAC is a machine-wide independent location that allows Strong-Named assemblies to register.
- It is a location where 2 versions of the same component can exist.
- **GAC Location**

« Windows > Microsoft.NET > assembly ▼ ↺					Search assembly
☐	Name	^	Date modified	Type	Size
	 GAC_32		18-11-2022 17:29	File folder	
	 GAC_64		07-05-2022 11:31	File folder	
	 GAC_MSIL		18-11-2022 17:47	File folder	

- Four components that make up an assembly are:
 1. Assembly Metadata
 2. Microsoft Intermediate Language (MSIL)
 3. Type Metadata
 4. Resources

Inside an Assembly



- The information about the assembly is called *assembly metadata*. It is also known as an *assembly manifest*. Assembly metadata contains the following information:
 - The assembly's name
 - Version number
 - Digital signature that uniquely identifies an assembly's creator.
 - All files which build up the assembly
 - Information regarding all of the referenced assemblies.
 - Information of all the exported classes, methods, properties, and other items.

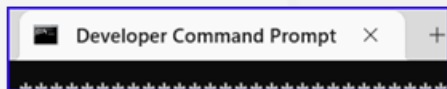
- MSIL plays a key role in making the .NET a language-neutral technology. Whenever you run the .NET application, the code is compiled down to the same MSIL, no matter what programming language is used. It is the MSIL that makes .NET capable of running independently. With the magic of MSIL, it becomes possible to enable programmers to “write once, run anywhere”.
- **Type Metadata**
- The *type* of an assembly is described by its *type metadata*. The resources and types that are publicly accessible are also called type metadata. The items, such as image files and other items required by the application, are referred to as *resources*. In the next section, we will get familiar with the different resources an assembly can have.

- Resources in .NET assembly can be easily placed from the assembly itself or an external resource. The process of accessing resources within an assembly is very straightforward. There are some important classes for dealing with the resources within an assembly. They are listed below:
- **ResourceManager:** A program within an assembly used to access and manage available resources.
- **ResourceWriter:** A program to write resource information to an external source.
- **ResourceReader:** A program within an assembly that reads the material from different resources, including the Internet.

- **Managed Code:** Managed code is the code that executed directly by CLR instead of the operating system.
- The language compiler compiles the managed code to IL or MSIL.
- This code does not depend on the machine configuration and can be executed on a different machine. Managed code process is as follows:
- *Choose Language Compiler --> Compile to MSIL, MSIL to Native Code, then Execute the code*

- **Unmanaged Code:**
- Unmanaged code is the code that is executed directly by the operating system outside the CLR environment.
- It is directly compiled to native machine code which depends on the machine configuration.
- In unmanaged code, the allocation of memory, Type safety, and security are required to be taken care of by the developer.
- If unmanaged code is not properly handled it may result in a memory leak.

- You can use the Ildasm.exe (IL Disassembler) to view Microsoft intermediate language (MSIL) information in a file.
- If the file being examined is an assembly, this information can include the assembly's attributes and references to other modules and assemblies.
- This information can be helpful in determining whether a file is an assembly or part of an assembly and whether the file has references to other modules or assemblies.
- To display the contents of an assembly using Ildasm.exe, enter `ildasm <assembly name>` at a Developer command prompt.



- For example, the following command disassembles the Hello.exe assembly.

```
C:\Program Files\Microsoft Visual Studio\2022\Community>ildasm.exe First_Project.exe
```

C:\Sneha\CDAC\Sept 22\DAC\C#\Pr...

File View Help

- C:\Sneha\CDAC\Sept 22\DAC\C#\PracticeCode\First_Project\bin\Debug
 - MANIFEST
 - AdoNetConsoleApplication
 - AdoNetConsoleApplication.Program1
 - First_Project
 - First_Project.DataStore`1<class T>
 - First_Project.Demo
 - First_Project.IMyInterface
 - First_Project.KeyValuePair`2<TKey,TValue>
 - First_Project.MyClass
 - First_Project.Person
 - Microsoft.CodeAnalysis
 - Microsoft.CodeAnalysis.EmbeddedAttribute
 - System.Runtime.CompilerServices

```

[assembly First_Project]
{
.ver 1:0:0:0

```

Prompt v17.4.1

MANIFEST
Find Find Next

```

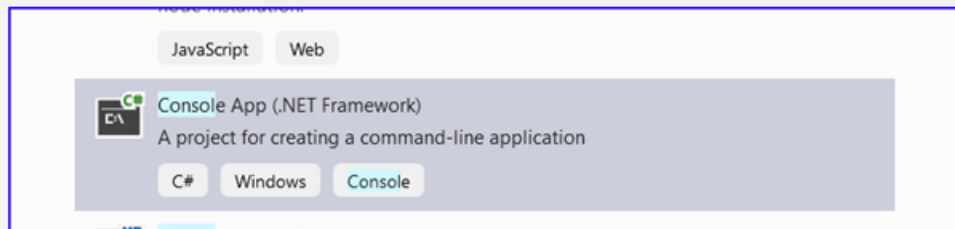
// Metadata version: v4.0.30319
[assembly extern System.Runtime]
{
  .publickeytoken = (B0 3F 5F 7F 11 D5 0A 3A ) // .
  .ver 6:0:0:0
}
[assembly extern System.Data.SqlClient]
{
  .publickeytoken = (B0 3F 5F 7F 11 D5 0A 3A ) // .
  .ver 4:6:1:5
}
[assembly extern System.Data.Common]
{
  .publickeytoken = (B0 3F 5F 7F 11 D5 0A 3A ) // .
  .ver 6:0:0:0
}
[assembly extern System.Console]
{
  .publickeytoken = (B0 3F 5F 7F 11 D5 0A 3A ) // .
  .ver 6:0:0:0
}

```

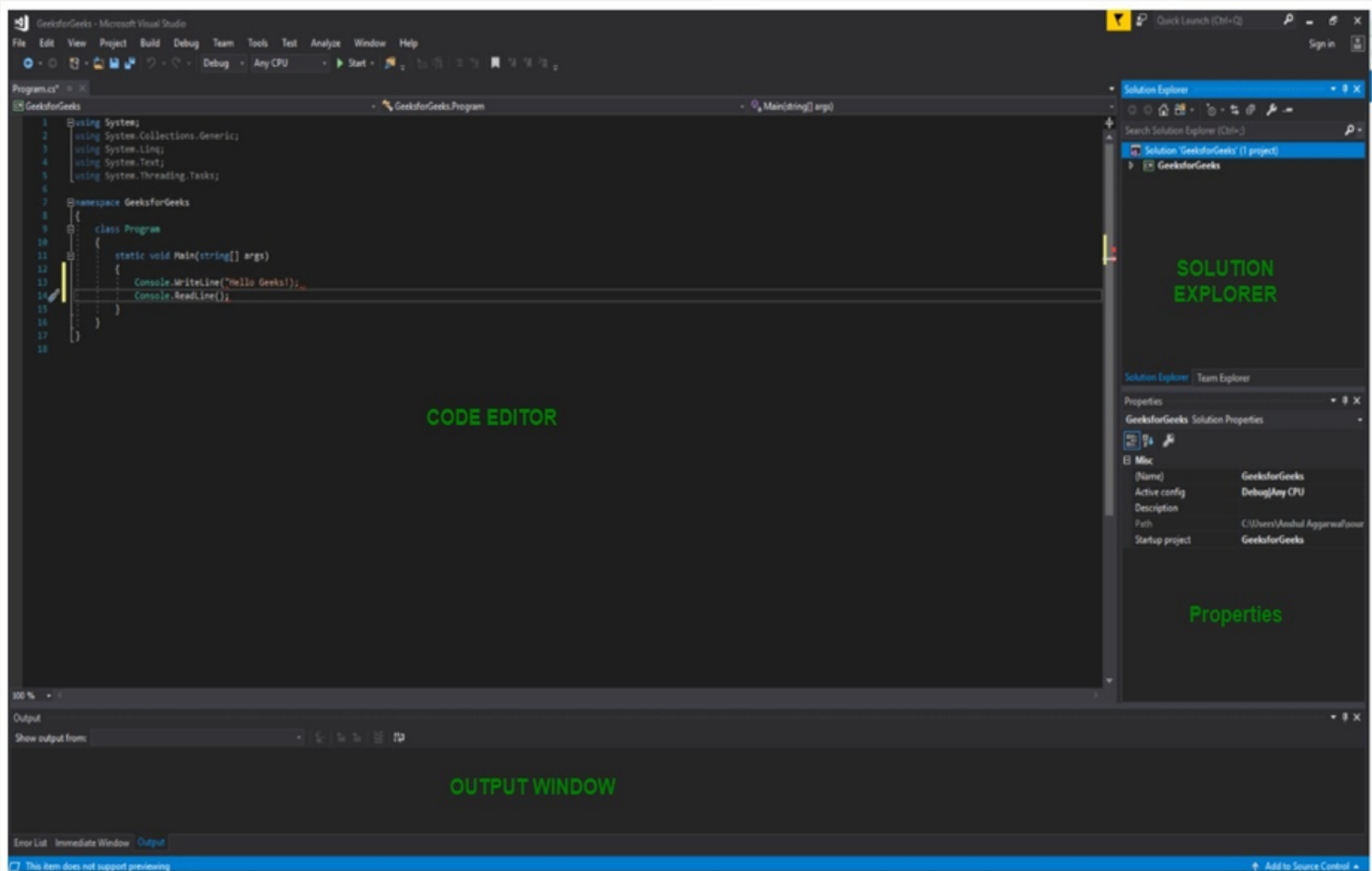
- In the .NET framework, an application domain (also known as an "AppDomain") is a unit of isolation for running .NET code. It is essentially a lightweight version of a process, and it provides a way to run multiple applications in a single process without those applications interfering with each other.
 1. Each AppDomain has its own set of assemblies, application-specific data, and security context.
 2. This allows multiple applications to run in the same process while still being isolated from each other.
 3. AppDomains can also be created and destroyed dynamically, which makes them useful for scenarios where you want to load and unload applications or components on demand.

- Here are some examples of when you might use AppDomains:
 1. Running multiple versions of the same application in the same process, with each version running in its own AppDomain.
 2. Loading and unloading plugins or extensions for an application, with each plugin running in its own AppDomain.
 3. Sandboxing untrusted code to prevent it from accessing resources or affecting other parts of the system.
- You can create and manipulate AppDomains using the AppDomain class in the System namespace.
- You can also create AppDomains using the AppDomain.CreateDomain method, which allows you to specify the configuration and security settings for the new AppDomain.

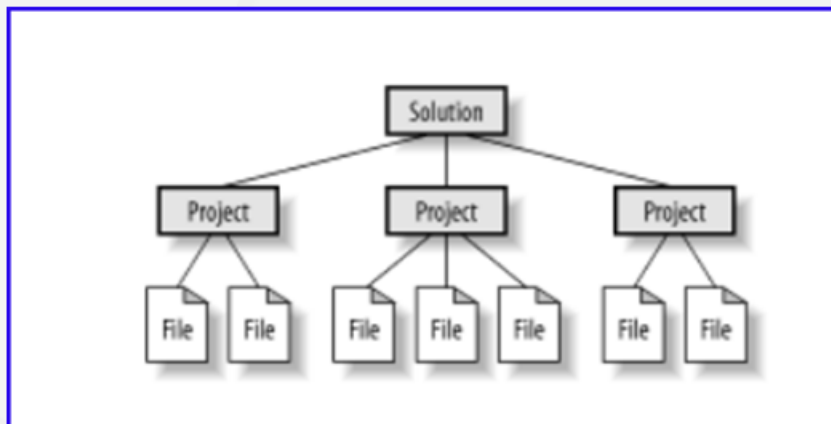
- **Create a .NET console application using Visual Studio**
- Create a .NET console app project named "HelloWorld".
 1. Start Visual Studio 2022.
 2. On the start page, choose **Create a new project**.
 3. On the **Create a new project** page, enter **console** in the search box. Next, choose **C#** from the language list, and then choose **All platforms** from the platform list. Choose the **Console Application** template(.Net Framework) , and then choose **Next**.



4. In the **Configure your new project** dialog, enter **HelloWorld** in the **Project name** box. Then choose **Create**.



- In C#, a solution is a container that holds one or more projects. A solution is a way to organize the files in your application and manage the build process.
- A solution can contain multiple projects, each of which can contain multiple files. The files in a project can include code files, resource files, and other types of files such as images and data files.



- A C# solution typically has the following structure:
- **Solution:** The top-level container that holds the projects in your application.
- **Project:** A project is a container for the files that make up a specific part of your application. A solution can contain multiple projects.
- **Code files:** Code files contain C# source code and are typically stored in the *.cs file extension.
- **Resource files:** Resource files contain non-code assets such as images, data files, and other types of resources.
- **References:** References are used to link a project to external assemblies or libraries that it uses.

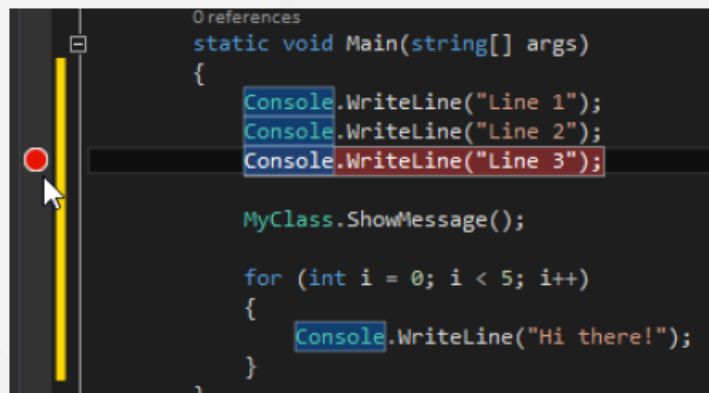
```
MySolution
|
+-- MyProject1
|   |
|   +-- MyClass1.cs
|   +-- MyClass2.cs
|   +-- MyResource1.resx
|
+-- MyProject2
|
|   +-- MyClass3.cs
|   +-- MyClass4.cs
|   +-- MyResource2.resx
```

- Debugging is the process of finding errors during application execution. It does not mean syntax errors, with which the application cannot be compiled, but on logic errors.

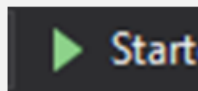
- **Debug Background code**

The basic tool for achieving Debug application code in the Visual Studio development environment is break points.

Break points are set by clicking on the margin on the left side of the code in the appropriate line.



- the Start Debugging option from the Debug menu, or simply the F5 key.



(Start Debug)

- After running, Debug execution will be stopped in the line where the breakpoint is defined.
- Further management of execution can be done by using the Step Into(F11), Step Over (F10) and Step Out (Shift+ F11) keys.



- If we want to get into the definition of some of the methods that are being invoked during debugging, the Step Into command can be used.

- Introduction to .Net Framework
- Versions of the Framework
- CLR
- Intermediate Language (IL)
- JIT Compilation
- Garbage Collection
- AppDomain Management
- CLS
- CTS
- FCL
- Using ILDASM
- Assemblies and their structure
- shared assembly
- Managed and Unmanaged Code
- Introduction to Visual Studio
- Debugging



Thank You